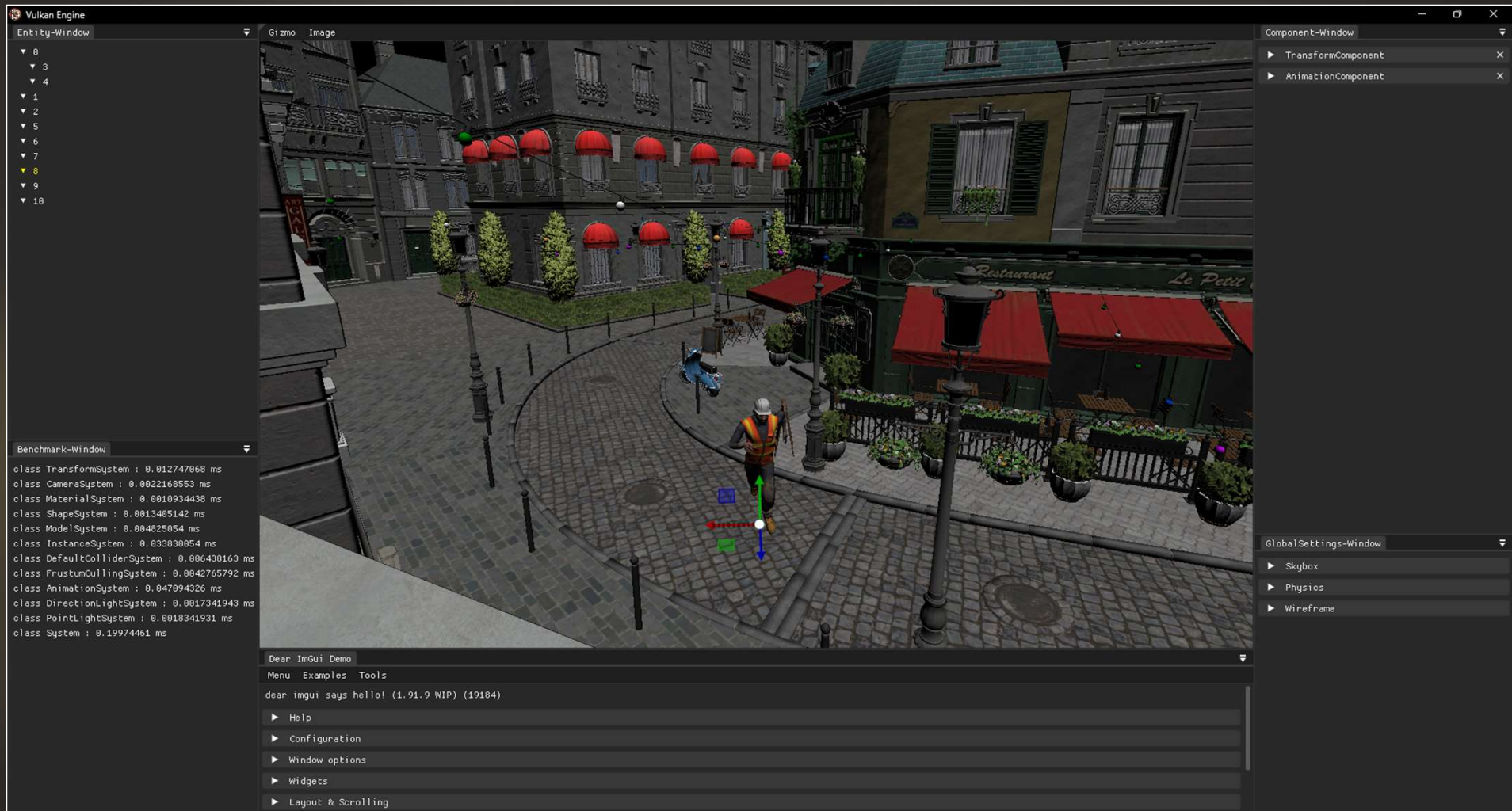




3D grafikus motor fejlesztése **Vulkan** alapokon

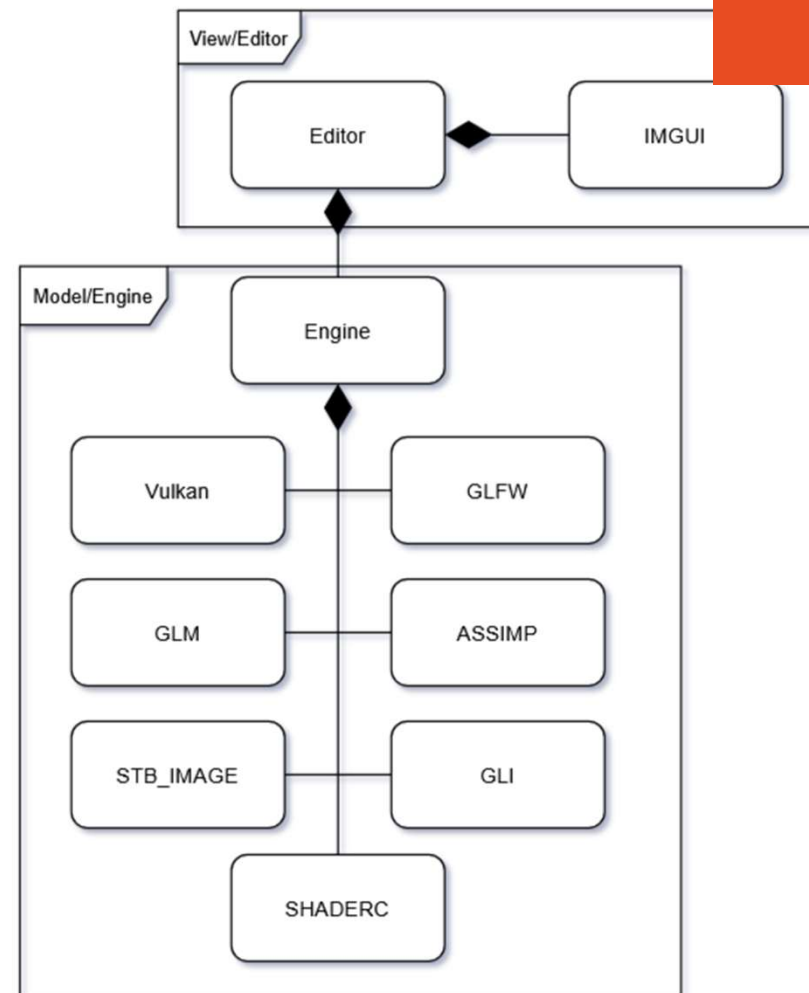
Készítette: Tamás Péter

Játékmotor futás közben



Játékmotor felépítése és felhasznált külső könyvtárak

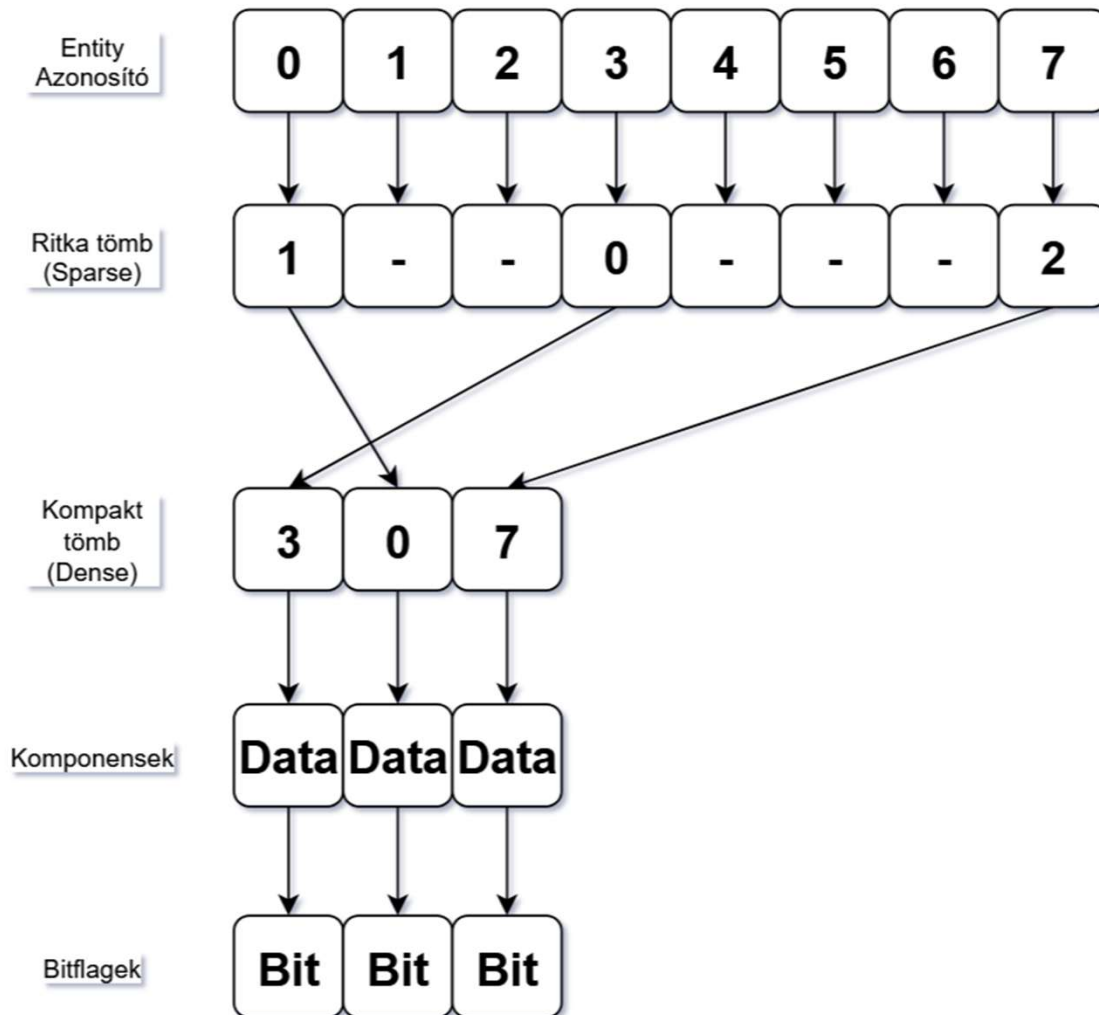
- ▶ **Editor**: futtatható állomány (exe)
- ▶ **Engine**: játékmotor (dll)
- ▶ **Vulkan**: Renderelés
- ▶ **ImGui**: Interfész megjelenítése
- ▶ **Glfw**: Ablak létrehozása
- ▶ **Glm**: Matematika
- ▶ **Assimp**: Model és animáció betöltés
- ▶ **Stb_Image**: Egyszerű textúrák betöltése
- ▶ **Gli**: dds textúrák betöltése
- ▶ **Shaderc**: futás idejű GLSL shader fordítás



Entity-Component-System I. (Adatorientált változat)

- ▶ **Entitások:** unsigned int azonosítók (UINT32_MAX = invalid entity)
- ▶ **Komponensek:** tisztán adatot tartalmazó struktúrák (nincs viselkedés)
- ▶ **Systemek:** adott típusú komponenseket frissítenek, és definiálnak nekik tetszőleges viselkedést.
- ▶ **Adatorientáltság előnyei:**
 - ▶ Adott típusú komponens összes előfordulását **egy vektorban kompakt** módon, egymás mellett szorosan tárolom. -> **Maximum cache lokalitás**
 - ▶ **Sparse Set** adatstruktúra **valós konstans $O(1)$** komponens elérést biztosít indirekt indexelésen keresztül
 - ▶ Systemek **teljesen párhuzamosan** egymástól függetlenül tudnak komponenseket frissíteni.
 - ▶ Azonos típusú komponensek **tökéletesen fragmentáltak**, így azok **párhuzamos SIMD** szerű frissítése is könnyen megvalósítható.
 - ▶ Minden komponens szükséges adatai ugyan olyan formában, kompaktan feltölthető **GPU bufferekbe** is

Entity-Component-System II. (Sparse Set és Data Pool)




```

auto LaunchSystemUpdateAsync = [&<typename T>() -> void {
    futures[Unique::TypeID<T>()] = std::async(std::launch::async,
        [&]() -> void {
            UpdateSystem<T>(frameIndex, deltaTime);
        });
};

LaunchSystemUpdateAsync.template operator() < TransformSystem > ();
LaunchSystemUpdateAsync.template operator() < CameraSystem > ();
LaunchSystemUpdateAsync.template operator() < AnimationSystem > ();

futures[Unique::TypeID<AnimationSystem>()].get();
LaunchSystemUpdateAsync.template operator() < ModelSystem > ();

futures[Unique::TypeID<ShapeSystem>()].get();
futures[Unique::TypeID<ModelSystem>()].get();
LaunchSystemUpdateAsync.template operator() < DefaultColliderSystem > ();

futures[Unique::TypeID<CameraSystem>()].get();
futures[Unique::TypeID<DefaultColliderSystem>()].get();
LaunchSystemUpdateAsync.template operator() < FrustumCullingSystem > ();

futures[Unique::TypeID<FrustumCullingSystem>()].get();
LaunchSystemUpdateAsync.template operator() < InstanceSystem > ();

for (auto& [_, future] : futures) {
    if (future.valid())
        future.get();
}

//... more systems, but these are to most important

std::for_each(std::execution::par,
    transformPool->GetDenseIndices().begin(),
    transformPool->GetDenseIndices().end(),
    [&](const Entity& entity) -> void {
        auto& transformComponent = transformPool->GetData(entity);

        //Calculate transform: 5x matrix multiplication
        //Calculate transform IT: 1x matrix multiplication

        transformPool->SetBit<CHANGED_BIT>(entity);
        transformComponent.version++;
    }
);

```

Entity-Component-System II. (Systemek és párhuzamosíthatóság)

Aszinkron managerek

- ▶ **Manager: ObjectPool** tervmintát követi, mindent 1x töltök be és tárolok el
- ▶ **ModelManager**: Minden egyes még nem betöltött modelt aszinkron külön szálon töltök be.
- ▶ **AnimationManager**: Minden egyes még nem betöltött animációt aszinkron külön szálon kezdek el betölteni.
- ▶ **ImageManager**: Minden textúrát külön szálon kezdek el feldolgozni és betölteni.
- ▶ **Kihívások:**
 - ▶ Ha minden textúra `vkCmdQueueSubmit2` hív az adatok feltöltéséhez gpu-ra, nagyon belassulna. -> **Batched Upload**
 - ▶ Textúrák mipmap generálásához `vkCmdBlitImage2` használom, amihez **graphics queue** szükséges. -> **Minimális akadás lesz ha sok textúrát tölt be egyszerre**

Model megjelenítése

- ▶ Sponza jelenet
- ▶ ~200.000 vertex
- ▶ ~800.000 index
- ▶ 26 material
- ▶ 43 image textúra



Animáció megjelenítése

- ▶ Három különböző animáció, közös model
- ▶ ~34.000 vertex
- ▶ ~160.000 index
- ▶ 52 csont
- ▶ 74 elemű node hierarchia (transzformáció)



```

auto& boneProcesInfos = animationComponent.animation->GetBoneProcessInfo();
auto& nodeProcessInfos = animationComponent.animation->GetNodeProcessInfo();
auto& nodeTransformInfos = modelComponent.model->GetNodeTransformInfos();
for (uint32_t i = 0; i < nodeProcessInfos.size(); ++i)
{
    animationComponent.nodeTransforms[i].localTransform = nodeTransformInfos[i].localTransform;

    if (nodeProcessInfos[i].boneIndex != UINT32_MAX)
    {
        auto& boneProcessInfo = boneProcesInfos[nodeProcessInfos[i].boneIndex];
        animationComponent.nodeTransforms[i].localTransform = boneProcessInfo.bone.GetTransform(animationComponent.time);
        animationComponent.nodeTransforms[i].nodeTransform.transform = animationComponent.nodeTransforms[i].localTransform * boneProcessInfo.offsetMatrix;
    }

    if (nodeProcessInfos[i].parentIndex != UINT32_MAX)
    {
        auto& parentTransform = animationComponent.nodeTransforms[nodeProcessInfos[i].parentIndex].localTransform;
        animationComponent.nodeTransforms[i].localTransform = parentTransform * animationComponent.nodeTransforms[i].localTransform;
        animationComponent.nodeTransforms[i].nodeTransform.transform = parentTransform * animationComponent.nodeTransforms[i].nodeTransform.transform;
    }

    animationComponent.nodeTransforms[i].nodeTransform.transformIT = glm::transpose(glm::inverse(animationComponent.nodeTransforms[i].nodeTransform.transform));
}

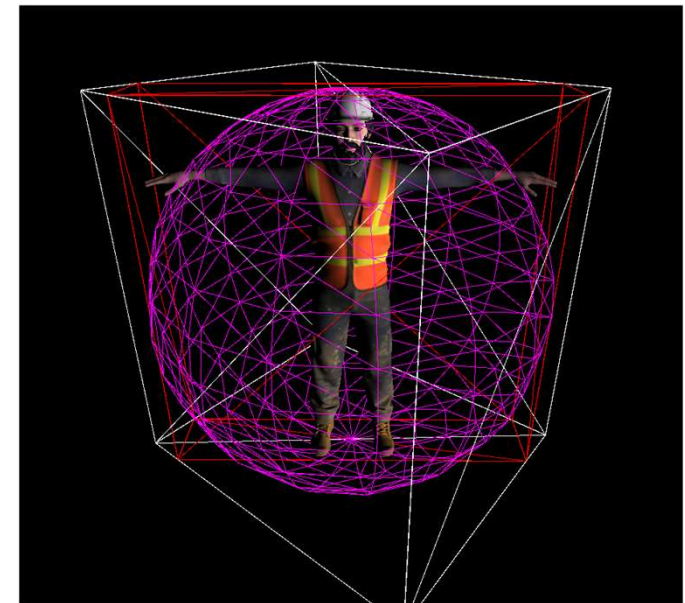
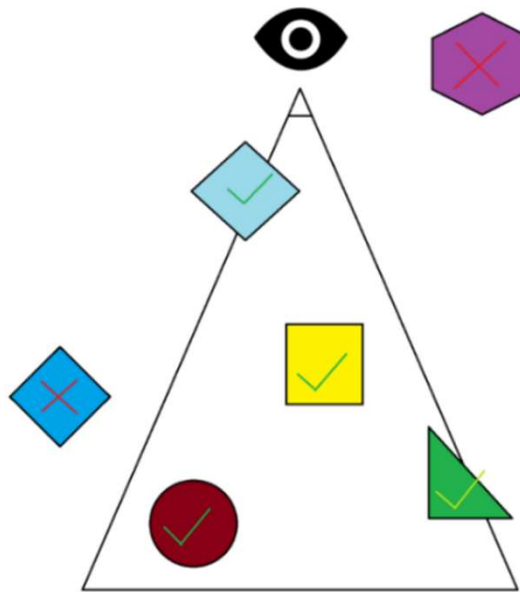
animationPool->ResetBit<UPDATE_BIT>(entity);
animationPool->SetBit<CHANGED_BIT>(entity);

```

Animációs mátrixok feldolgozása (Iterációs megközelítés)

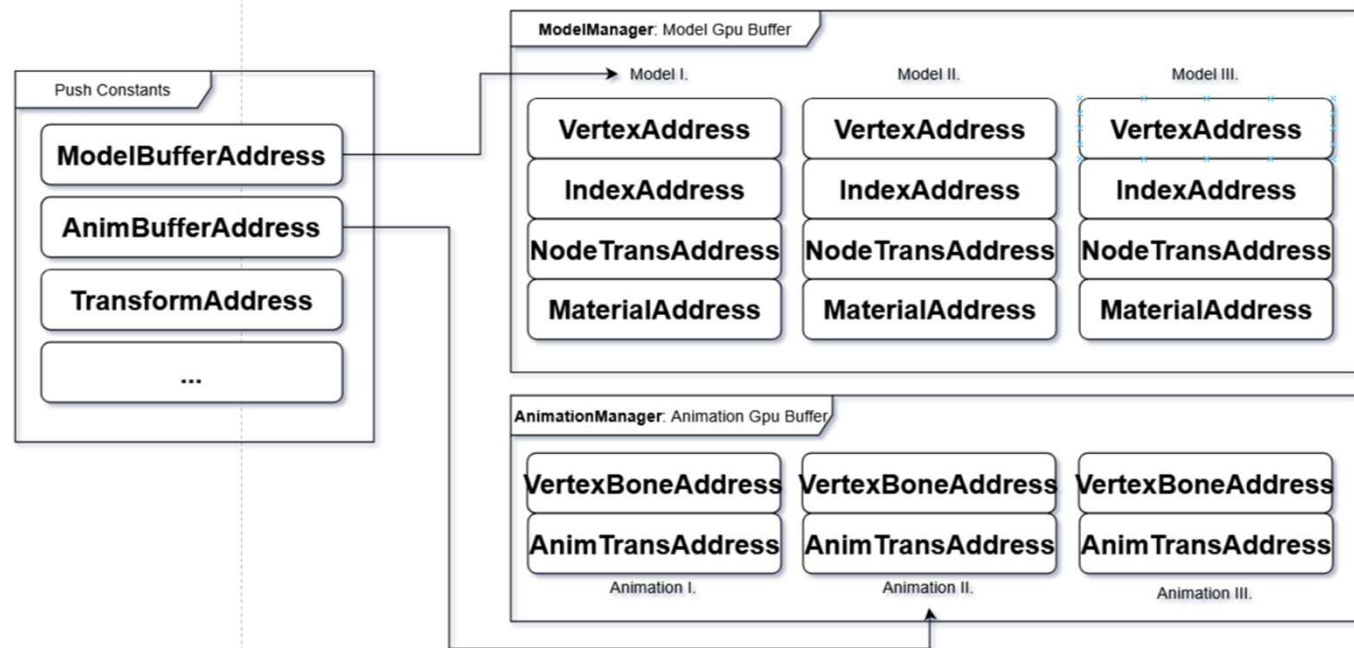
Optimalizációs megoldások (Frustum Culling)

- ▶ AABB collider
- ▶ OBB collider
- ▶ Sphere collider
- ▶ Ezeket automatikusan generálja az engine
- ▶ **Camera frustum vs AABB** algoritmus van implementálva
- ▶ Maga az entity culling **pont és spotfényforrások** árnyékgenerálását is fel fogja gyorsítani



Bindless Vulkan

- ▶ **Device Address:** GPU bufferekhez generálok GPU pointer addresseket, amiket utána shaderbe feltöltök. (Buffer Linked List megvalósítható)
- ▶ **Dynamic Descriptor Indexing:** Kb. OpenGL bindless textúrák, egy nagy descriptor set van, és minden betöltött image egy adott indexhez fog tartozni.



Deferred Renderelés

- ▶ Fekete textúrából indul a light pass, és az irány, pont és spot fényforrások által kiszámolt fényhatást hozzáadom a textúrához (**One-One blending**)
- ▶ **Optimalizáció:** pont és spot fényforrás kiterjedése jól meghatározható, így ott gömb és kúp alakzatot renderelek -> **Rengeteg fragment nem fog aktiválódni**

